

HW4_Penguins_Pandas

May 21, 2026

1 HW4: Palmer Penguins — Summary Statistics & Plots

1.0.1 Setup

- Use only pandas, matplotlib, and seaborn.
- Do your work in the cells provided; you may add extra cells.

```
[310]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# (Optional) nicer display for DataFrames
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 3)
```

```
[311]: ### Load the dataset
penguins = pd.read_csv("penguins.csv")

# Standardize column names (snake_case)
penguins.columns = [c.strip().lower().replace(' ', '_') for c in penguins.
                    ↪columns]

penguins.head()
```

```
[311]: species      island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen         39.1           18.7           181.0
1  Adelie  Torgersen         39.5           17.4           186.0
2  Adelie  Torgersen         40.3           18.0           195.0
3  Adelie  Torgersen          NaN           NaN           NaN
4  Adelie  Torgersen         36.7           19.3           193.0

      body_mass_g      sex  year
0         3750.0    male  2007
1         3800.0  female  2007
2         3250.0  female  2007
3            NaN      NaN  2007
4         3450.0  female  2007
```

1.0.2 Part 1) Data Vitals

Answer the following questions.

Your answer to each question must include some code that produces the result. You can, but you do not have to use a complete sentence to answer each question. Add cells as needed.

- a. What is the shape of the data?
- b. What are the Column names and data types of each column?
- c. How many values in each column are missing?
- d. How many penguins are there of each species? How many male and female penguins are there?

```
[312]: # a. shape of the data
penguins.shape
```

```
[312]: (344, 8)
```

```
[313]: # b. column names and data types
penguins.dtypes
```

```
[313]: species          str
island              str
bill_length_mm     float64
bill_depth_mm      float64
flipper_length_mm  float64
body_mass_g        float64
sex                str
year               int64
dtype: object
```

```
[314]: # c. number of missing values in each column
penguins.isnull().sum()
```

```
[314]: species          0
island              0
bill_length_mm      2
bill_depth_mm       2
flipper_length_mm   2
body_mass_g         2
sex                 11
year                0
dtype: int64
```

```
[315]: # d. number of penguins in each species
penguins['species'].value_counts()
```

```
[315]: species
      Adelie      152
      Gentoo     124
      Chinstrap   68
      Name: count, dtype: int64
```

```
[316]: # d. number of female and male penguins
      penguins['sex'].value_counts()
```

```
[316]: sex
      male      168
      female    165
      Name: count, dtype: int64
```

1.0.3 Part 2) Handle missing data

- a. Create a copy `penguins_clean` that **drops** rows with missing values in `['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']`.
- b. Report the new shape of `penguins_clean`.

```
[317]: # a. create a copy of dataset that drops rows with missing values
      penguins_clean = penguins.dropna(
          subset =
      ↪ ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']
      )
      # show that there are no missing values
      penguins_clean.isnull().sum()
```

```
[317]: species      0
      island      0
      bill_length_mm  0
      bill_depth_mm  0
      flipper_length_mm  0
      body_mass_g    0
      sex          0
      year         0
      dtype: int64
```

```
[318]: # b. new shape of penguins_clean
      penguins_clean.shape
```

```
[318]: (333, 8)
```

1.0.4 Part 3) Numeric summary statistics

- a. Compute the following for each numeric column: count, mean, std, min, 25%, 50%, 75%, max (hint: `.describe()`)

- b. Compute the mean and std for each numeric column for each species

```
[319]: # a. compute summary statistics for every numeric column
penguins_clean.describe()
```

```
[319]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
count	333.000	333.000	333.000	333.000	333.000
mean	43.993	17.165	200.967	4207.057	2008.042
std	5.469	1.969	14.016	805.216	0.813
min	32.100	13.100	172.000	2700.000	2007.000
25%	39.500	15.600	190.000	3550.000	2007.000
50%	44.500	17.300	197.000	4050.000	2008.000
75%	48.600	18.700	213.000	4775.000	2009.000
max	59.600	21.500	231.000	6300.000	2009.000

```
[320]: # b. compute mean and std for each numeric column for each species
num_cols = penguins_clean.select_dtypes(include='number').columns
penguins_clean.groupby('species')[num_cols].agg(['mean', 'std'])
```

```
[320]:
```

	bill_length_mm		bill_depth_mm		flipper_length_mm		
	mean	std	mean	std	mean	std	\
species							
Adelie	38.824	2.663	18.347	1.219	190.103	6.522	
Chinstrap	48.834	3.339	18.421	1.135	195.824	7.132	
Gentoo	47.568	3.106	14.997	0.986	217.235	6.585	

	body_mass_g		year		
	mean	std	mean	std	
species					
Adelie	3706.164	458.620	2008.055	0.812	
Chinstrap	3733.088	384.335	2007.971	0.863	
Gentoo	5092.437	501.476	2008.067	0.789	

1.0.5 Part 4) Two-way grouped summaries

a) For each (**species**, **sex**) combination, compute the **mean** and **count** of:

- bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g.

b) Which (species, sex) has the **largest average body__mass_g**? Show the row.

```
[321]: # a. compute mean and count for each (species, sex) combination
cols = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']

grouped_summary = penguins_clean.groupby(['species', 'sex'])[cols].agg(['mean', 'count'])
grouped_summary
```

```
[321]:
```

		bill_length_mm		bill_depth_mm		flipper_length_mm		
		mean	count	mean	count	mean		
species	sex							
Adelie	female	37.258	73	17.622	73	187.795		
	male	40.390	73	19.073	73	192.411		
Chinstrap	female	46.574	34	17.588	34	191.735		
	male	51.094	34	19.253	34	199.912		
Gentoo	female	45.564	58	14.238	58	212.707		
	male	49.474	61	15.718	61	221.541		

		body_mass_g			
		count	mean	count	
species	sex				
Adelie	female	73	3368.836	73	
	male	73	4043.493	73	
Chinstrap	female	34	3527.206	34	
	male	34	3938.971	34	
Gentoo	female	58	4679.741	58	
	male	61	5484.836	61	

```
[322]: # b. (species, sex) with largest average body_mass_g
grouped_summary.loc[[grouped_summary[('body_mass_g', 'mean')].idxmax()]]
```

```
[322]:
```

		bill_length_mm		bill_depth_mm		flipper_length_mm		
		mean	count	mean	count	mean	count	
species	sex							
Gentoo	male	49.474	61	15.718	61	221.541	61	

		body_mass_g		
		mean	count	
species	sex			
Gentoo	male	5484.836	61	

1.0.6 Part 5) Correlations

- a. Compute the **correlation matrix** among numeric columns.
- b. Compute the correlation matrix **within each species** (hint: groupby).
- c. Find an example of Simpson's Paradox in the data.
https://en.wikipedia.org/wiki/Simpson%27s_paradox
 - An example Simpson's Paradox would be a pair of variables that have a positive correlation within subgroups, but when all the groups are combined, there is negative correlation (or vice-versa).

```
[323]: # a. compute correlation matrix for numeric columns
penguins_clean.corr(numeric_only=True)
```

```
[323]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	\
bill_length_mm	1.000	-0.229	0.653	
bill_depth_mm	-0.229	1.000	-0.578	
flipper_length_mm	0.653	-0.578	1.000	
body_mass_g	0.589	-0.472	0.873	
year	0.033	-0.048	0.151	

	body_mass_g	year
bill_length_mm	0.589	0.033
bill_depth_mm	-0.472	-0.048
flipper_length_mm	0.873	0.151
body_mass_g	1.000	0.022
year	0.022	1.000

```
[324]: # b. compute correlation matrix within each species
penguins_clean.groupby('species').corr(numeric_only=True)
```

```
[324]:
```

		bill_length_mm	bill_depth_mm	flipper_length_mm	\
species					
Adelie	bill_length_mm	1.000	0.386	0.332	
	bill_depth_mm	0.386	1.000	0.311	
	flipper_length_mm	0.332	0.311	1.000	
	body_mass_g	0.544	0.580	0.465	
	year	0.011	-0.235	0.328	
Chinstrap	bill_length_mm	1.000	0.654	0.472	
	bill_depth_mm	0.654	1.000	0.580	
	flipper_length_mm	0.472	0.580	1.000	
	body_mass_g	0.514	0.604	0.642	
	year	0.042	-0.059	0.346	
Gentoo	bill_length_mm	1.000	0.654	0.664	
	bill_depth_mm	0.654	1.000	0.711	
	flipper_length_mm	0.664	0.711	1.000	
	body_mass_g	0.667	0.723	0.711	
	year	0.205	0.243	0.201	

		body_mass_g	year
species			
Adelie	bill_length_mm	0.544	0.011
	bill_depth_mm	0.580	-0.235
	flipper_length_mm	0.465	0.328
	body_mass_g	1.000	-0.046
	year	-0.046	1.000
Chinstrap	bill_length_mm	0.514	0.042
	bill_depth_mm	0.604	-0.059
	flipper_length_mm	0.642	0.346
	body_mass_g	1.000	0.037
	year	0.037	1.000

Gentoo	bill_length_mm	0.667	0.205
	bill_depth_mm	0.723	0.243
	flipper_length_mm	0.711	0.201
	body_mass_g	1.000	0.052
	year	0.052	1.000

```
[325]: # c. example of simpson's paradox in dataset
# negative overall correlation between bill_length_mm and bill_depth_mm
penguins_clean[['bill_length_mm', 'bill_depth_mm']].corr()
```

```
[325]:          bill_length_mm  bill_depth_mm
bill_length_mm      1.000      -0.229
bill_depth_mm      -0.229      1.000
```

```
[326]: # positive correlation within each species
penguins_clean.groupby('species')[['bill_length_mm', 'bill_depth_mm']].corr()
```

```
[326]:          bill_length_mm  bill_depth_mm
species
Adelie   bill_length_mm      1.000      0.386
         bill_depth_mm      0.386      1.000
Chinstrap bill_length_mm      1.000      0.654
         bill_depth_mm      0.654      1.000
Gentoo   bill_length_mm      1.000      0.654
         bill_depth_mm      0.654      1.000
```

An example of the Simpson's Paradox is the relationship between `bill_length_mm` and `bill_depth_mm`. Overall, the correlation between `bill_length_mm` and `bill_depth_mm` is negative (-0.229). However, there is a positive correlation within each species. Adelie is about 0.386, Chinstrap is about 0.654, and Gentoo is about 0.654. The direction of the relationship changes after being grouped by species.

1.1 Plots

- Each figure must have a **title**, **axis labels**, and a **legend** (when appropriate).
- Use readable tick labels and sensible limits.
- Be sure each graphic is displayed.

1.1.1 Part 6) Histograms of body mass by species

Create **separate** histograms of `body_mass_g` for each species (three panels).

- Use the same binning across panels so comparisons are fair.
- Label axes clearly

```
[327]: species_list = penguins_clean['species'].unique()

bins = range(
```

```

int(penguins_clean['body_mass_g'].min()),
int(penguins_clean['body_mass_g'].max()) + 100,
100
)

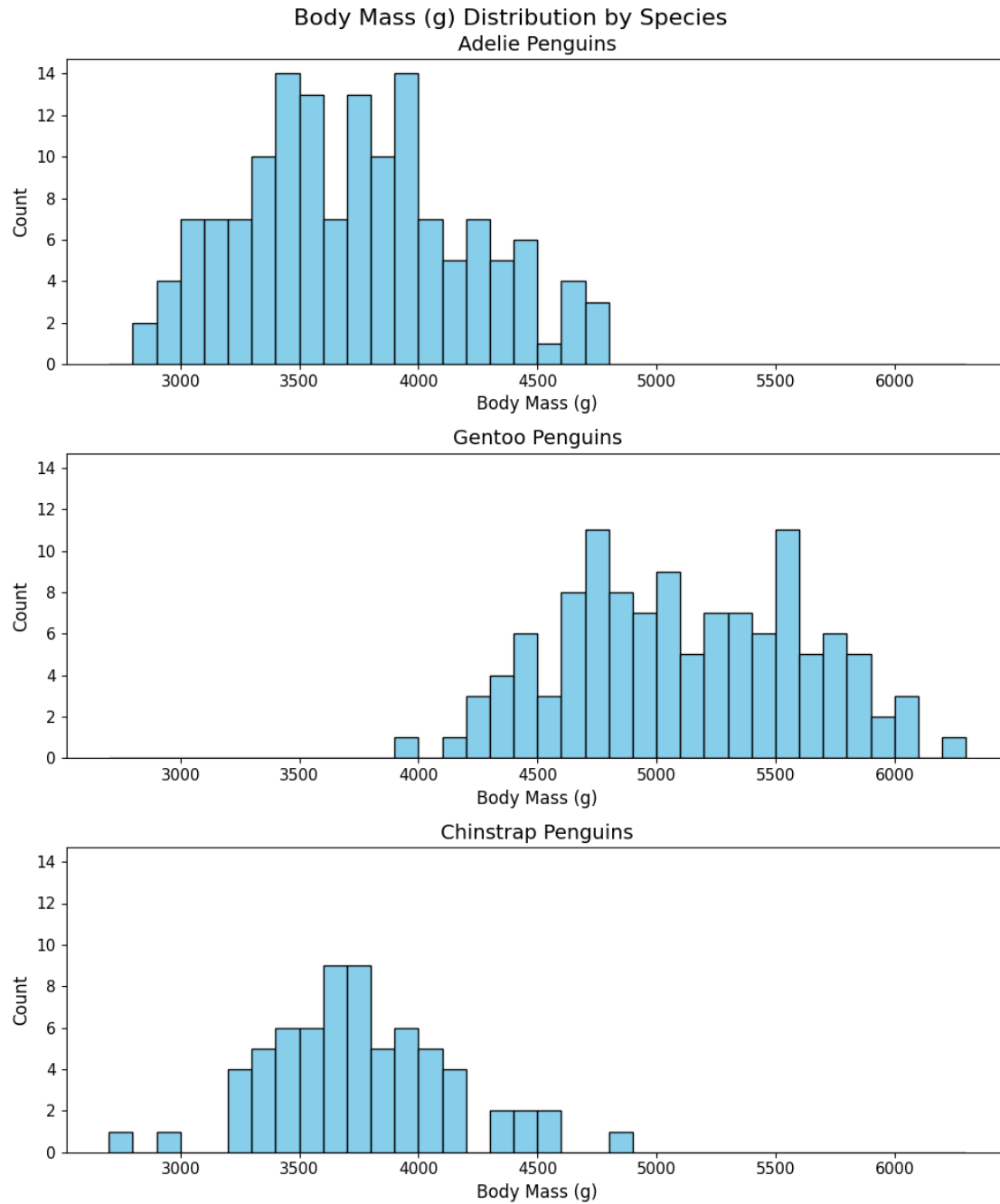
fig, axes = plt.subplots(3, 1, figsize=(10, 12), sharex=True, sharey=True)

for ax, species in zip(axes, species_list):
    species_data = penguins_clean[penguins_clean['species'] == species]

    ax.hist(species_data['body_mass_g'], bins=bins, edgecolor='black',
    color='skyblue')
    ax.set_title(f'{species} Penguins', fontsize=14)
    ax.set_xlabel('Body Mass (g)', fontsize=12)
    ax.set_ylabel('Count', fontsize=12)
    ax.tick_params(axis='both', labelsize=11)
    ax.tick_params(axis='x', labelbottom=True)

plt.suptitle('Body Mass (g) Distribution by Species', fontsize=16)
plt.tight_layout()
plt.show()

```



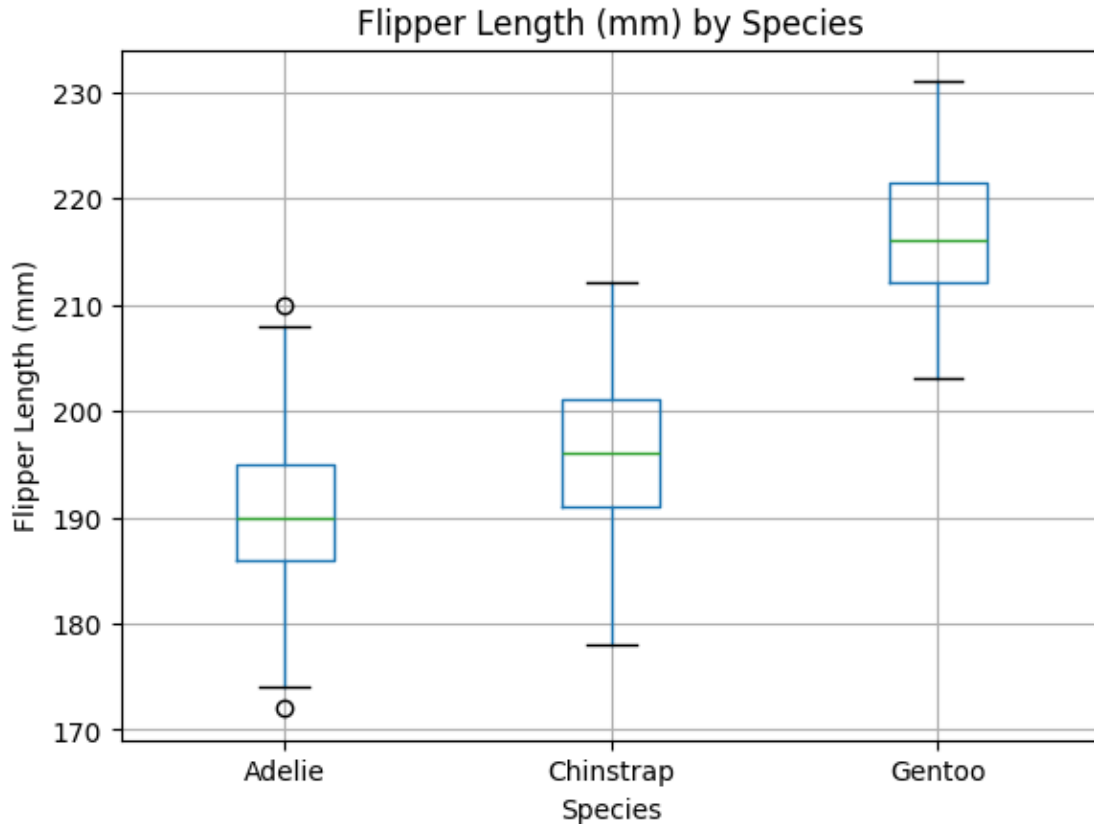
1.1.2 Part 7) Boxplots of flipper length

Make a **single figure** with boxplots of `flipper_length_mm` by **species**.

Briefly compare the medians and spreads in 2–3 sentences in a markdown cell.

```
[328]: fig, ax = plt.subplots()

penguins_clean.boxplot(column='flipper_length_mm', by='species', ax=ax)
ax.set_title('Flipper Length (mm) by Species')
ax.set_xlabel('Species')
ax.set_ylabel('Flipper Length (mm)')
plt.suptitle('')
plt.show()
```

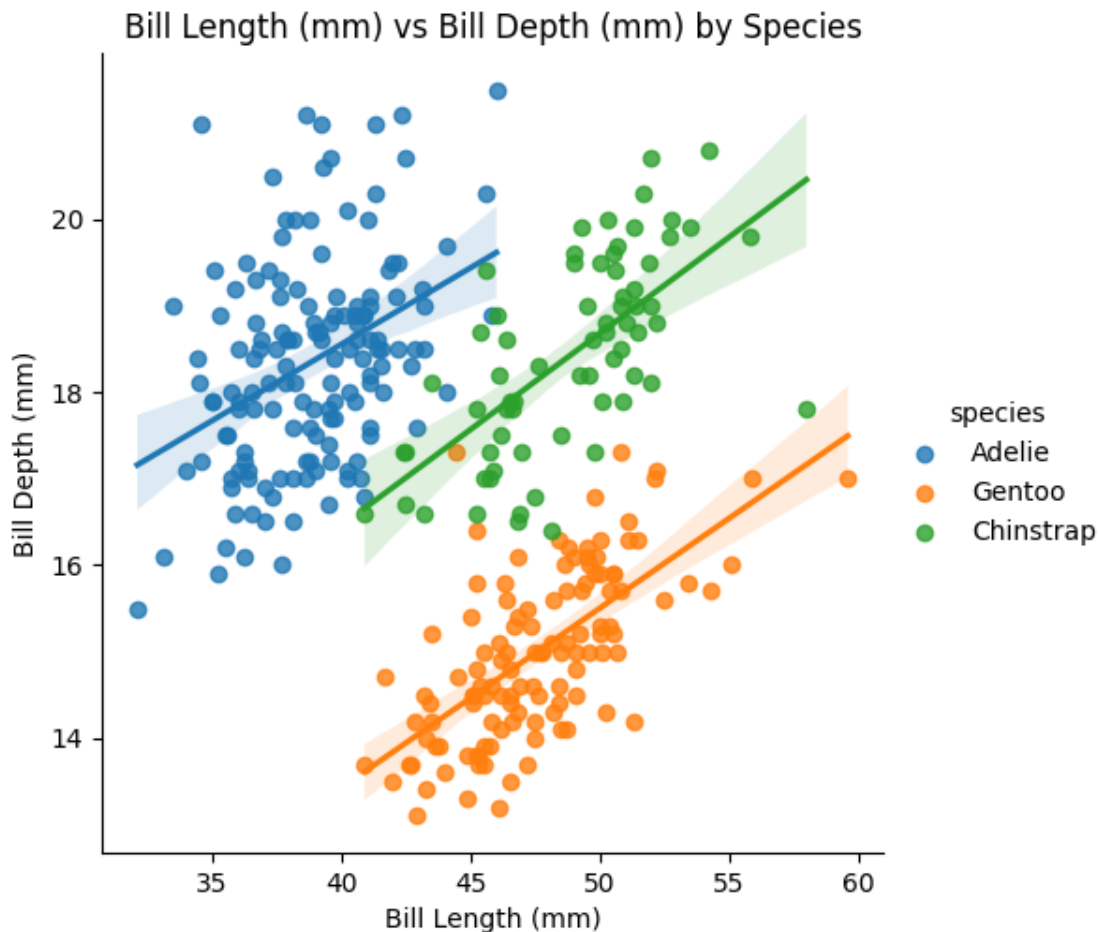


Gentoo penguins have the highest median flipper length, followed by Chinstrap, then Adelie, with Chinstrap and Adelie being much closer to each other. Adelie and Chinstrap also have similar spreads, while Gentoo has a slightly smaller spread and generally bigger flipper lengths. Adelie also has a couple of outliers.

1.1.3 Part 8) Scatter: bill length vs bill depth

- Make a scatter plot of **bill_length_mm** (x) vs **bill_depth_mm** (y), colored by species.
- Add a simple **linear fit line** per species
- Include a legend

```
[329]: sns.lmplot(data=penguins_clean, x='bill_length_mm', y='bill_depth_mm',
    ↪hue='species')
plt.title('Bill Length (mm) vs Bill Depth (mm) by Species')
plt.xlabel('Bill Length (mm)')
plt.ylabel('Bill Depth (mm)')
plt.show()
```



1.1.4 Part 9) Stacked bars: sex proportions by species

Compute counts of **sex** within each **species**, convert to **proportions**, and make a **stacked bar chart**.

- Bars should be species on the x-axis with male/female proportions stacked to 1.0.

```
[330]: fig, ax = plt.subplots()

pd.crosstab(penguins_clean['species'], penguins_clean['sex'],
    ↪normalize='index').plot(kind='bar', stacked=True, ax=ax)
```

```
ax.set_title('Sex Proportions by Species')
ax.set_xlabel('Species')
ax.set_ylabel('Proportion')
ax.set_ylim(0, 1)
ax.legend(title='Sex')
plt.xticks(rotation=0)
plt.show()
```

